A

Project Report

Submitted in partial fulfilment of the requirements for the completion of the Course

EDS Lab Codetantra

Practical No.1

(2304102L)

Submitted by,

Harshvardhan Lokhande (202501090156)

Under the Guidance of,

Ms. Savita Mane(SM)

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Mechanical Engineering,

MIT Academy of Engineering,

Alandi road, Pune - 412 105,

Maharashtra (India)

1.1. Practice Lab Assignments

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum *p* is calculated using the formula:
p = m v

where:

p is the mass of the object (in kilograms).

m is the velocity of the object (in meters per second).

Input Format:

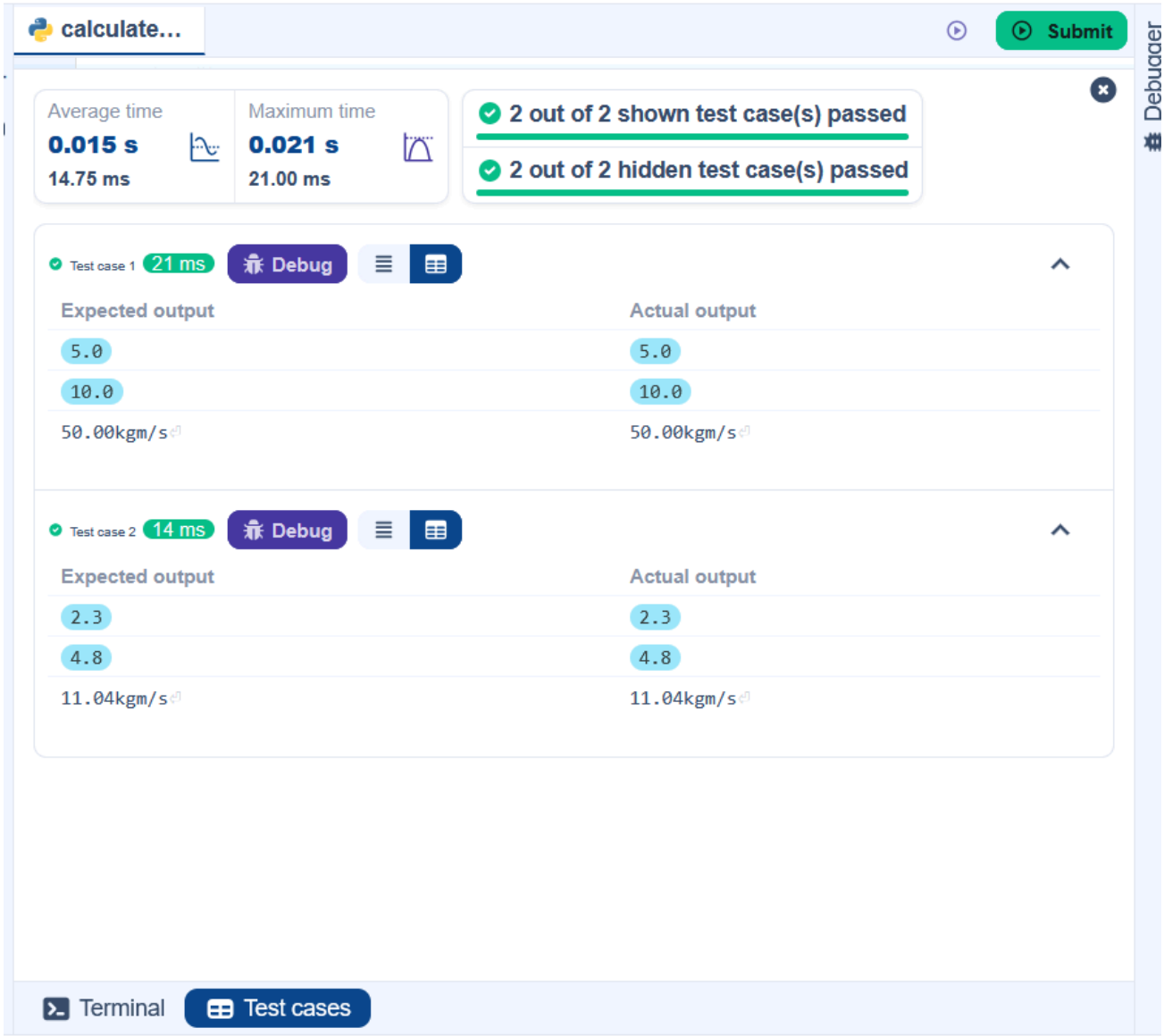
A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

```
mass=float(input())
velocity=float(input())
momentum=mass*velocity
print(f"{momentum:.2f}kgm/s")
```



1.1.2 Conditional Calculation based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in *n*.

Constraints:

$1 \leq n \leq 999$

Input Format: The input consists of a single integer *n*.

Output Format:

If *n* is a single-digit number, print its square.

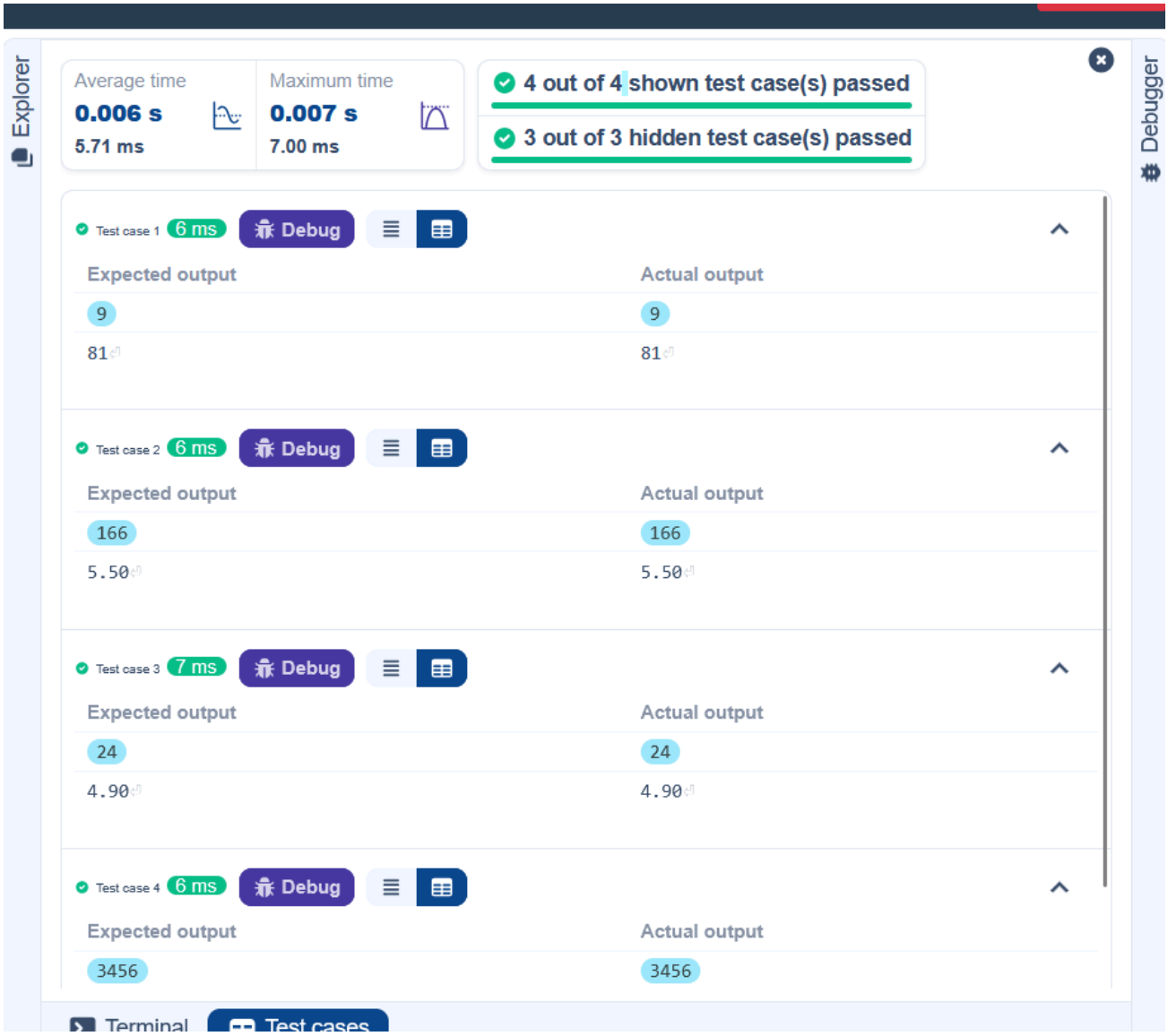
If *n* is a two-digit number, print its square root (rounded to two

decimal places).

If *n* is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

```
import math

number=int(input())

if 0<= number <=9:

        print(number**2)

elif 10<= number <=99:

        print(f"{math.sqrt(number):.2f}")

elif 100<= number <=999:

        print(f"{math.cbrt(number):.2f}")

else:

        print("Invalid")
```



1.1.3. Days Between Two Dates

Write a Python program to calculate the number of days between two dates.

Input Format:

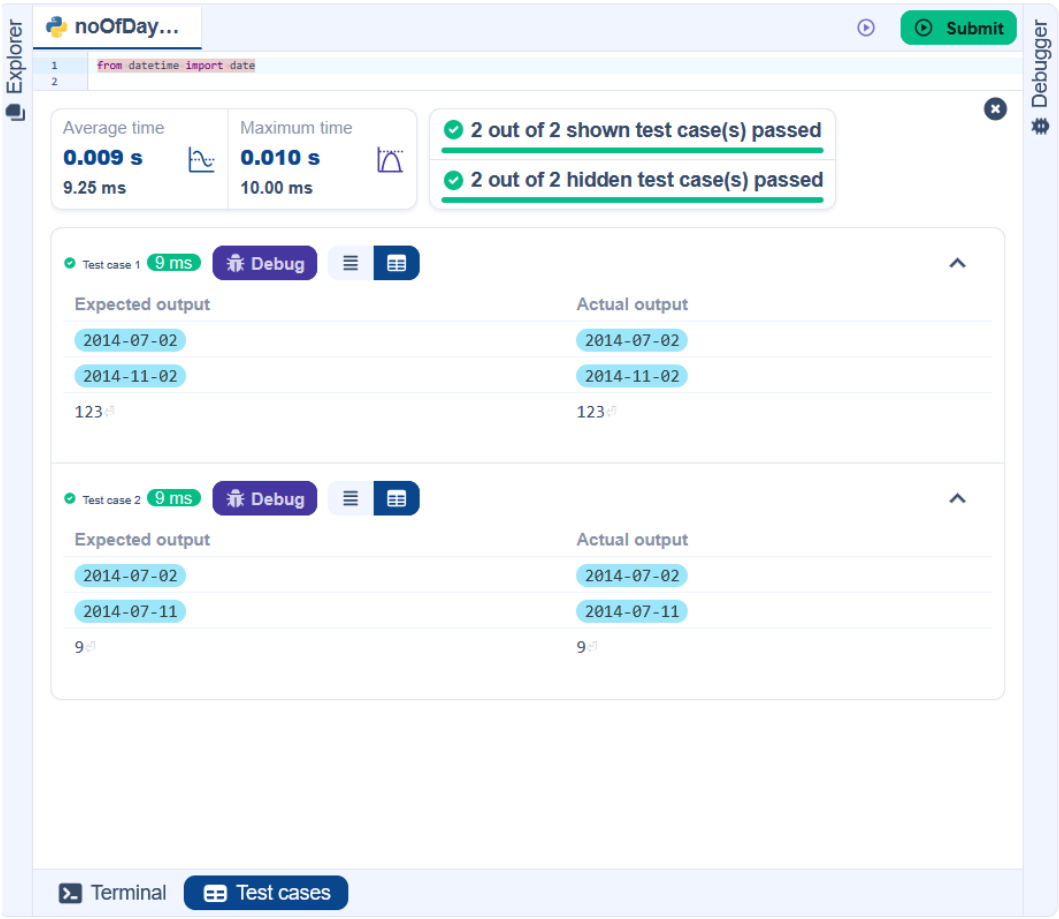
The first line contains the first date in the format *YYYY–MM–DD*.

The second line contains the second date in the format *YYYY–MM–DD*.

Output Format:

An integer representing the number of days between the given dates.

```
from datetime import datetime

date1=input().strip()
        date2=input().strip()

d1=datetime.strptime(date1, "%Y-%m-%d")
        d2=datetime.strptime(date2, "%Y-%m-%d")

difference=d2-d1
print(difference.days)
```



1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

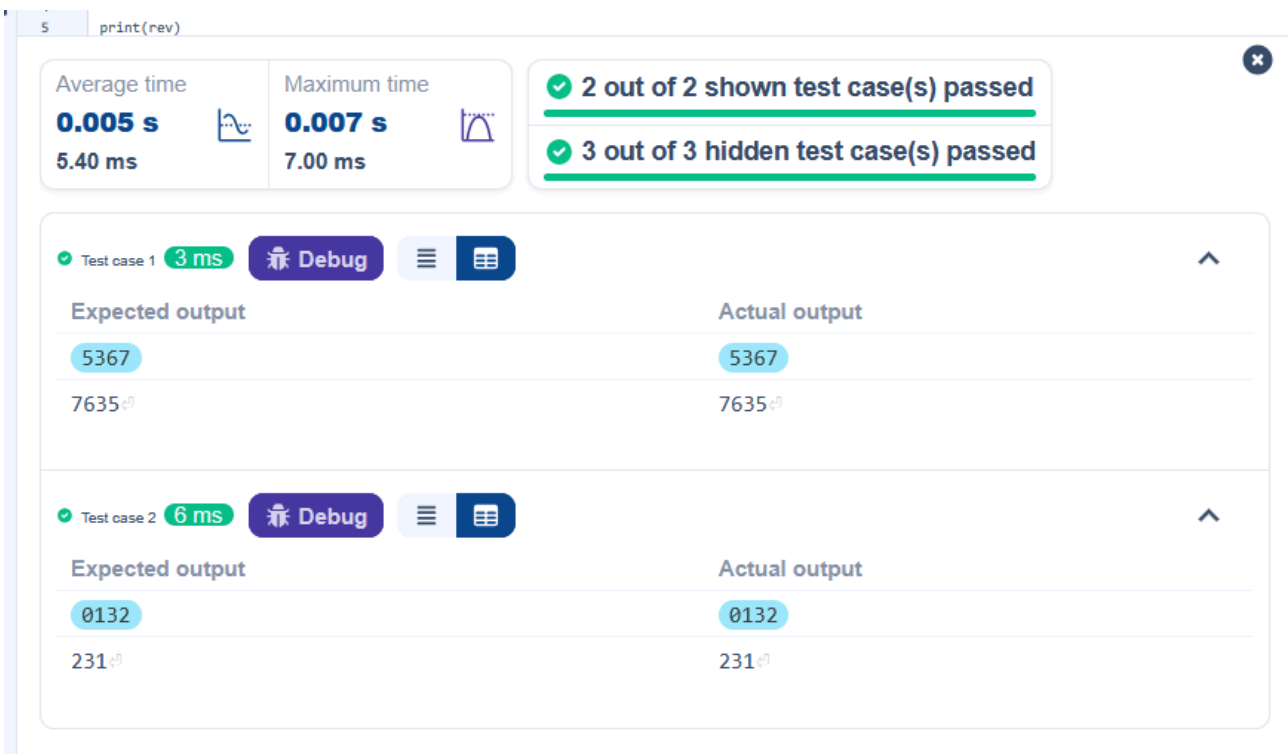
Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

```
n = int(input())

reversed_num=int(str(num)[::-1])

print(reversed_num)
```



1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number.
  <number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

```
num = int(input())

for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

Explorer

multiplica...

Submit

Debugger

```python
n=int(input())
for i in range(1,11):
    print(f"{n} x {i} = {n * i}")
```

Average time	Maximum time
0.004 s 4.25 ms	0.006 s 6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 6 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Test case 2 4 ms Debug

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40
5 x 9 = 45	5 x 9 = 45
5 x 10 = 50	5 x 10 = 50

Terminal Test cases

‹ Prev Reset Submit Next ›

1.2 Lab Assignments

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer *n*, the number of courses.
- The second input will be *n* integers representing the marks of the student in each of the *n* courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

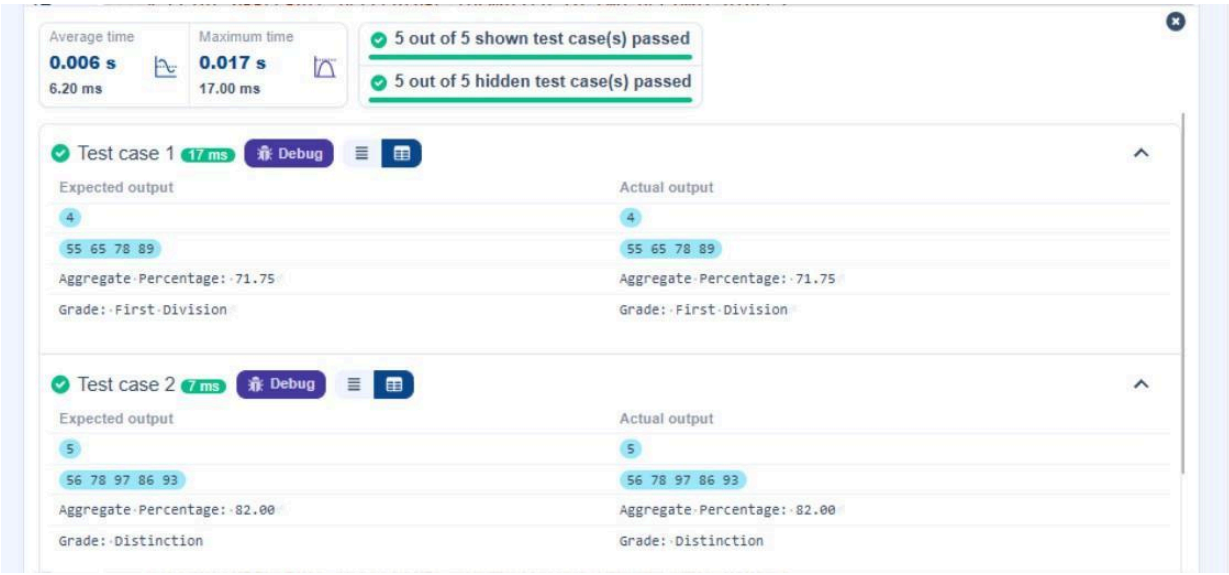
- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
n = int(input())
marks= list(map(int,input().split()))
failed= False
for m in marks:

    if m<40:
        failed=True
        break

if failed:
        print("Fail")
else:
        total= sum(marks)
        agg=total/n
        print("Aggregate Percentage: "f"{agg:.2f}")

        if agg > 75:
            print("Grade: Distinction")
        elif agg >=60:
            print("Grade: First Division")
        elif agg>=50:
            print("Grade: Second Division")
        elif agg >=40:
```

print("Grade: Third Division")



1.2.2. Fibonacci Series using Recursive Function

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

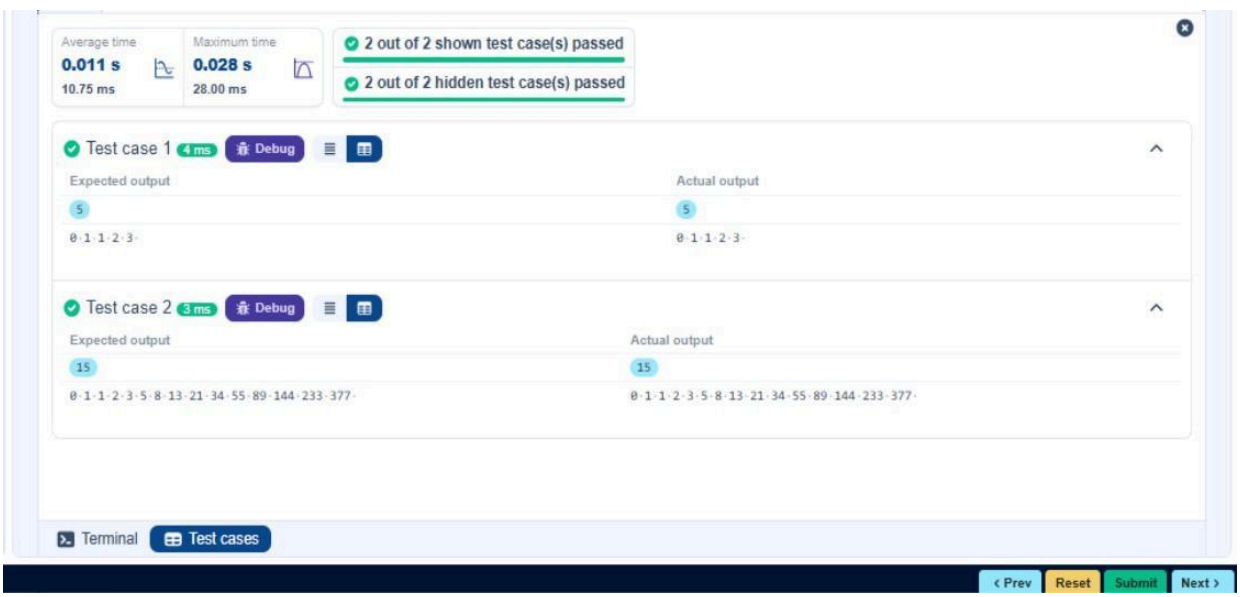
A single integer n representing the number of terms of generate.

Output Format:

A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

```
def fibonacci(n):
        if n==1:
                return 0
        elif n==2:
                return 1
        else:
            return fibonacci(n-1) + fibonacci(n-2)



n = int(input())
for i in range(1, n + 1):
    print(fibonacci(i), end=" ")
```

```
def fibonacci(n):
        if n==1:
                return
        0 elif n==2:
                return 1
        else:
                return fibonacci(n-1) + fibonacci(n- 2)
n = int(input())
for i in range(1, n + 1):
        print(fibonacci(i), end=" ")
```



1.2.3. Pattern-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Test case 1:

4

*·⏎

··⏎

··*·⏎

··*·*·⏎

```
n = int(input())
for i in range(1, n + 1):
    print('* ' * i)
```

Pattern-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Test case 1:

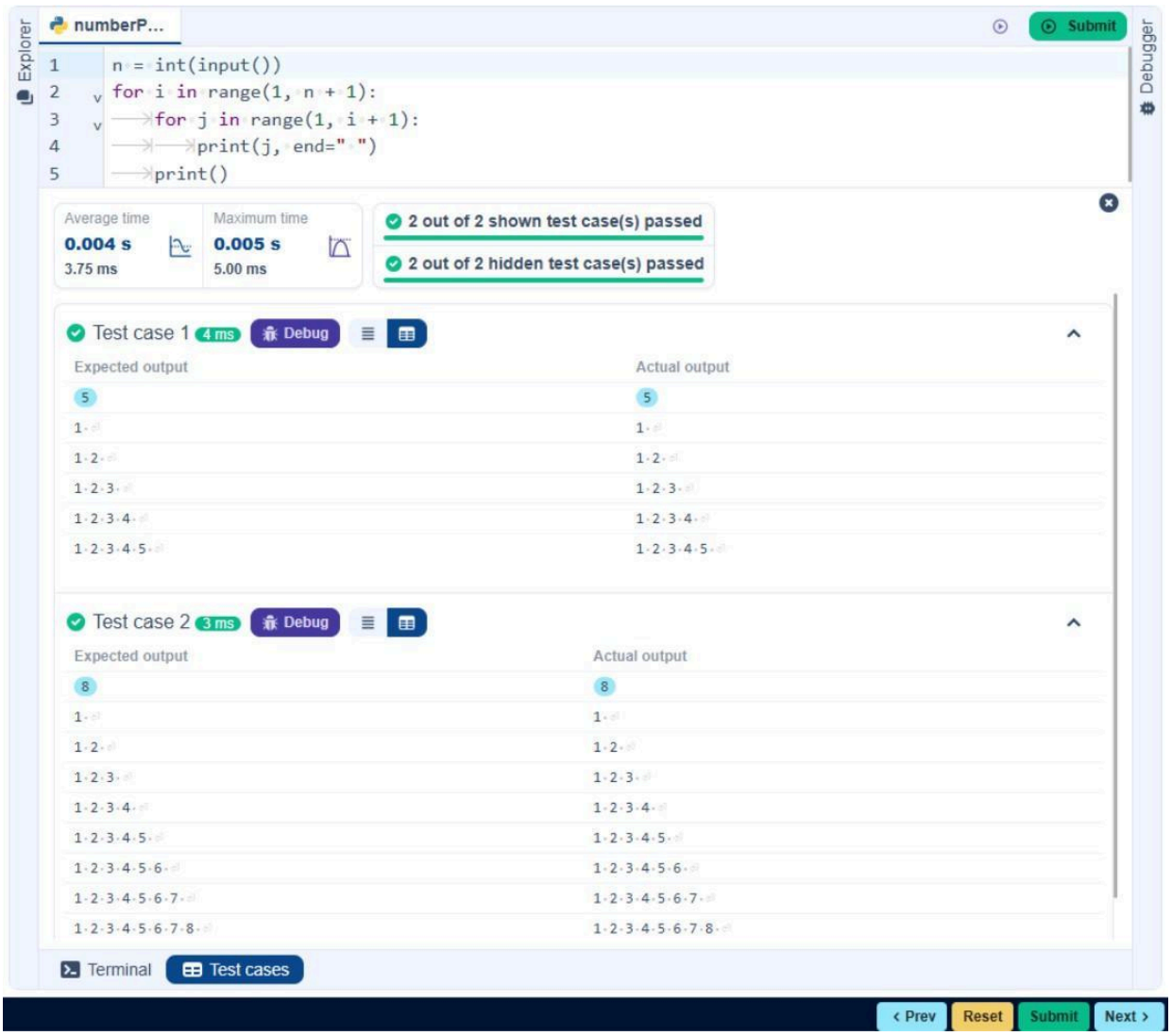
5

1·⏎

1·2·⏎

1·2·3·⏎

1·2·3·4·⏎

1·2·3·4·5·⏎

```
n = int(input())
for i in range(1, 1+n):
    for j in range(1, 1+i):
        print(j, end=" ")
    print("")
```